15-442/15-642: Machine Learning Systems

Data Layouts

Spring 2026

Tianqi Chen and Zhihao Jia

Carnegie Mellon University

Outline

Tiled & Thread Layout

Distributed Layout

Swizzle Layout

Outline

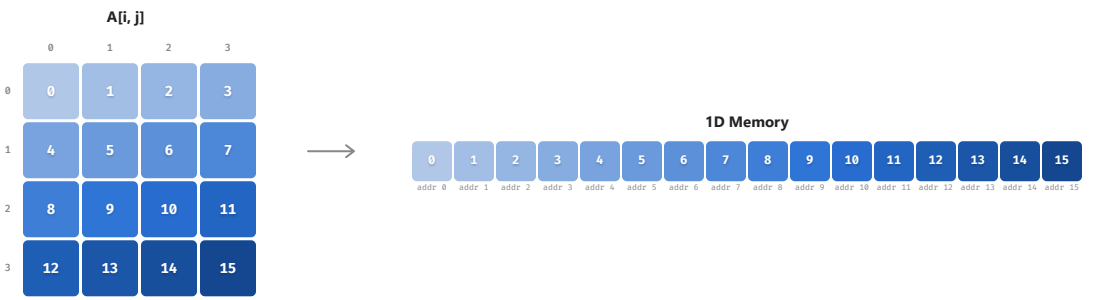
Tiled & Thread Layout

Distributed Layout

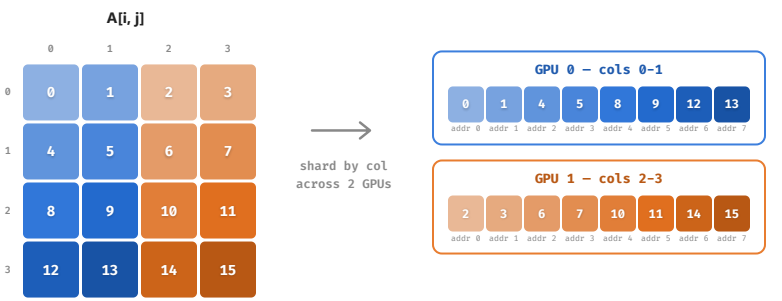
Swizzle Layout

What is Data Layout

Mapping from logical indices to the phyiscal location of the data



or different devices...

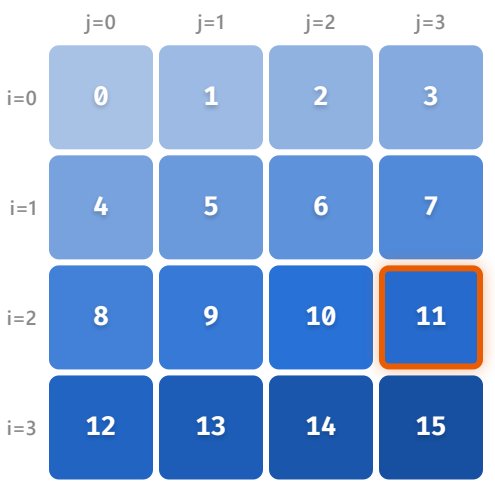


Shape–Stride Model

We will use this notation:

$$S[(4,\ 4)\ :\ (4,\ 1)]$$

shape stride



Address computation

$$addr(i, j) = i \times 4 + j \times 1$$

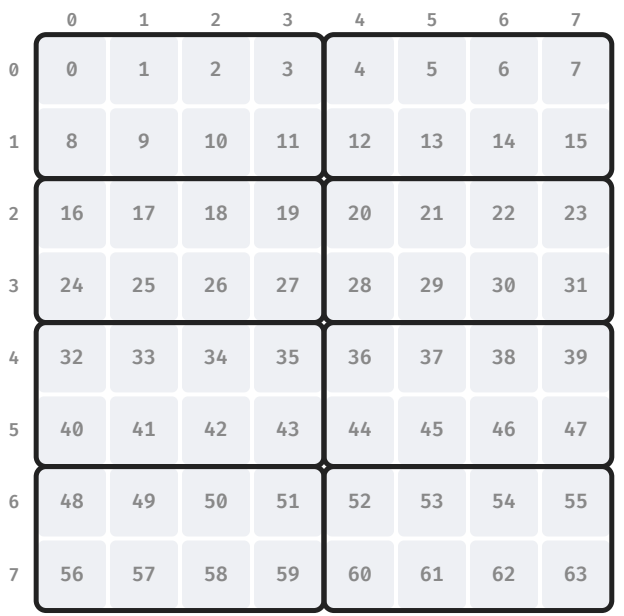
Example: $addr(2, 3) = 2 \times 4 + 3 \times 1 = 11$

This notation can be viewed as simplfied version of CuTe, with difference that we adopt row-major ordering.

Tile Layout

Representing tiling

Layout: S[(4, 2, 2, 4) : (16, 4, 8, 1)]



Address computation

Original shape (8 , 8)

↓ reshape

Tiled shape (4 , 2 , 2 , 4)

Mapped indices (i//2 , i%2 , j//4 , j%4)

addr = (i//2)×16+(i%2)×4+(j//4)×8+(j%4)×1

Example: addr(2, 3) = 1×16 + 0×4 + 0×8 + 3×1 = 19

Tile Layout: Interactive Example

S[(4, 2, 2, 4) : (16, 4, 8, 1)]

Logical view vs Physical memory | click cell to see mapping

Logical Matrix

	c0	c1	c2	c3	c4	c5	c6	c7
r0	0	1	2	3	4	5	6	7
r1	8	9	10	11	12	13	14	15
r2	16	17	18	19	20	21	22	23
r3	24	25	26	27	28	29	30	31
r4	32	33	34	35	36	37	38	39
r5	40	41	42	43	44	45	46	47
r6	48	49	50	51	52	53	54	55
r7	56	57	58	59	60	61	62	63

Physical Memory (tiled 2×4)

	+0	+1	+2	+3	+4	+5	+6	+7
0	0	1	2	3	8	9	10	11
8	4	5	6	7	12	13	14	15
16	16	17	18	19	24	25	26	27
24	20	21	22	23	28	29	30	31
32	32	33	34	35	40	41	42	43
40	36	37	38	39	44	45	46	47
48	48	49	50	51	56	57	58	59
56	52	53	54	55	60	61	62	63

Click a cell to see how logical element maps to physical memory

Named Axes in Strides

We need to represent different hardware axese:

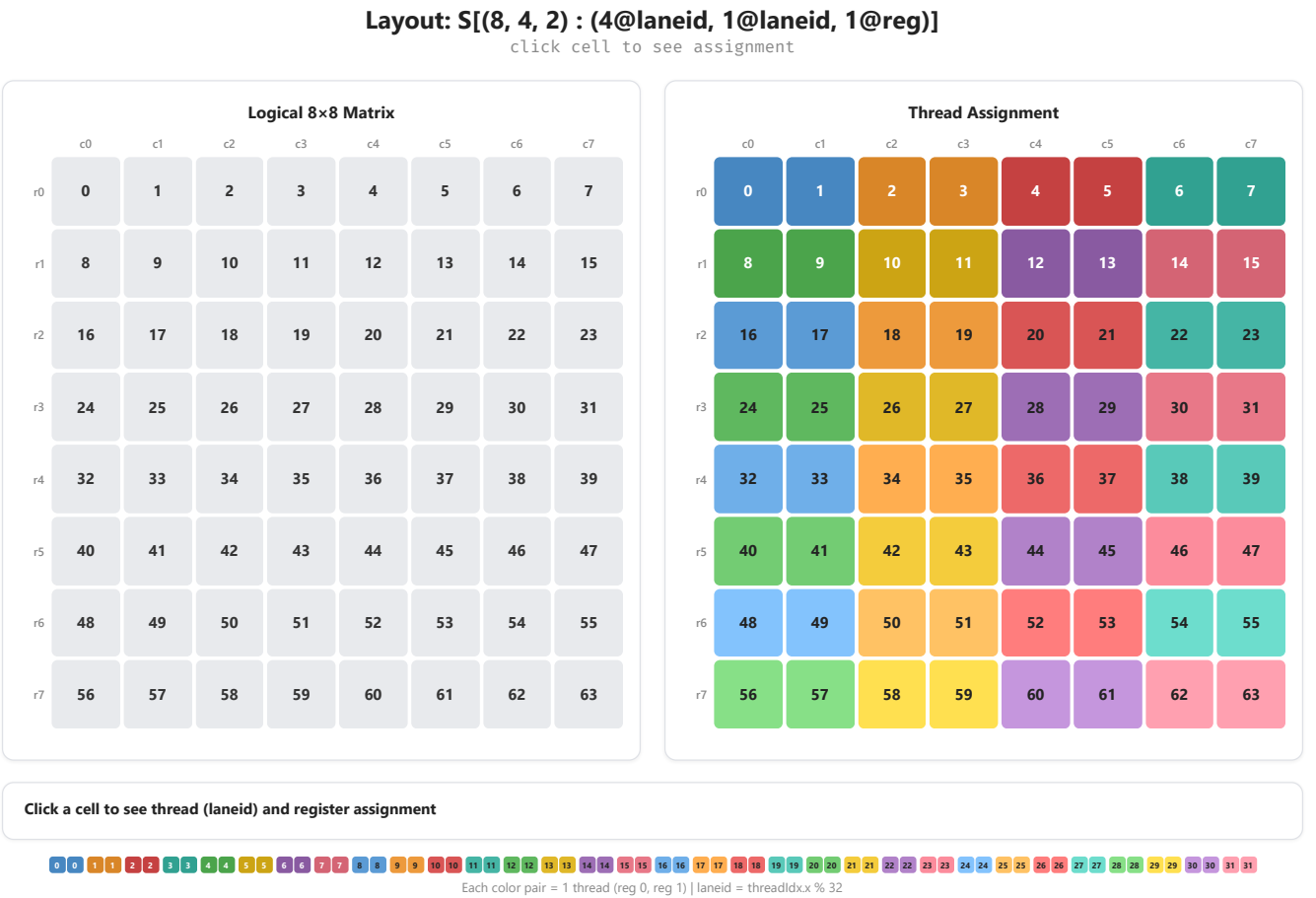
- Lane and col dimension in tensor memory
- Thread indcies

It is useful to have named axes, we use **@m** for normal meory

- `Row-major 8×16: S[(8, 16) : (16@m, 1@m)]`

The `@axis` tag can also be other things like `@warpid`, or `@tmemcol`

Named Axes Example: Thread + Register



Outline

Tiled & Thread Layout

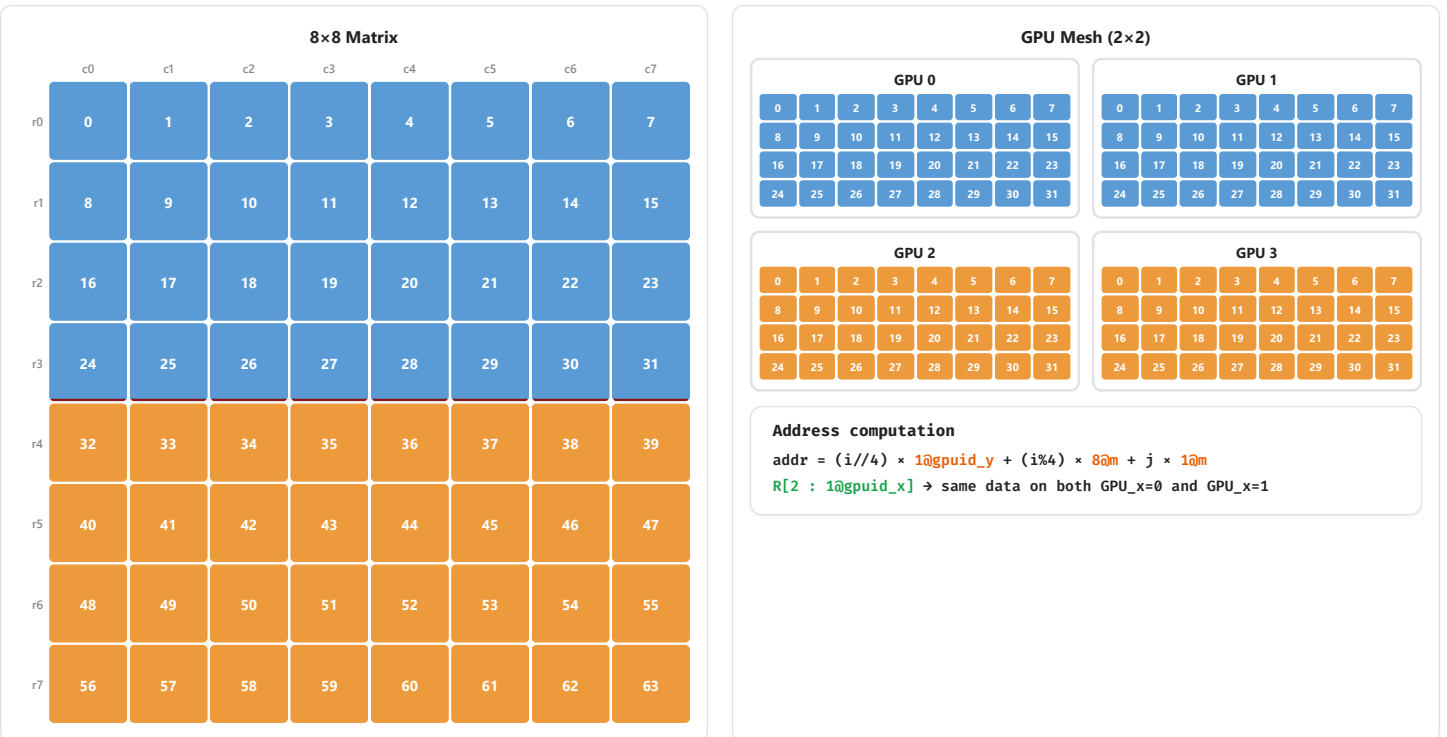
Distributed Layout

Swizzle Layout

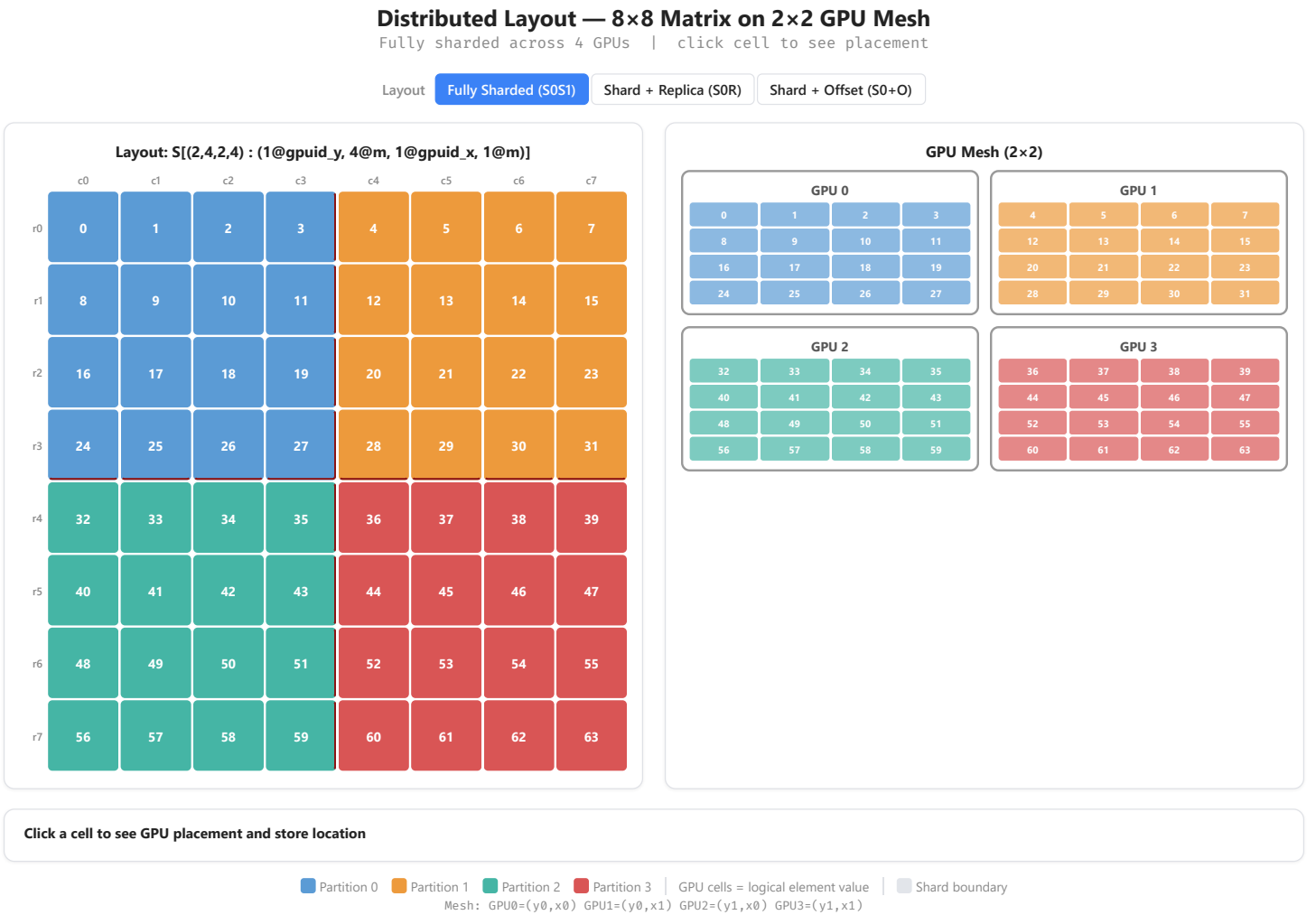
Distributed Axes: @gpuid_y, @gpuid_x

Representing broadcast / replication

Layout: S[(2, 4, 8) : (1@gpuid_y, 8@m, 1@m)] + R[2 : 1@gpuid_x]



Distributed Axes: Interactive



Outline

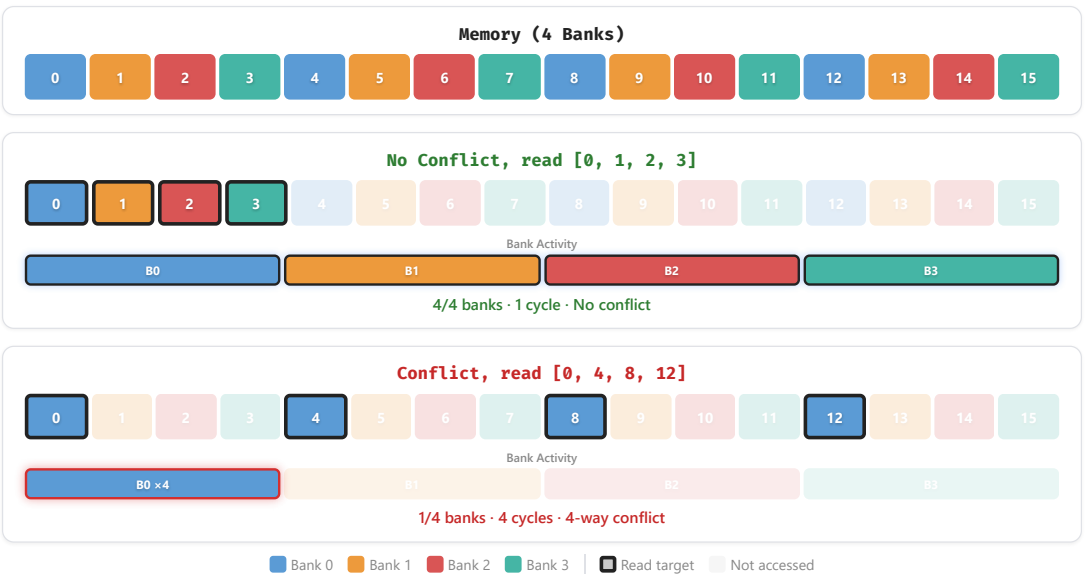
Tiled & Thread Layout

Distributed Layout

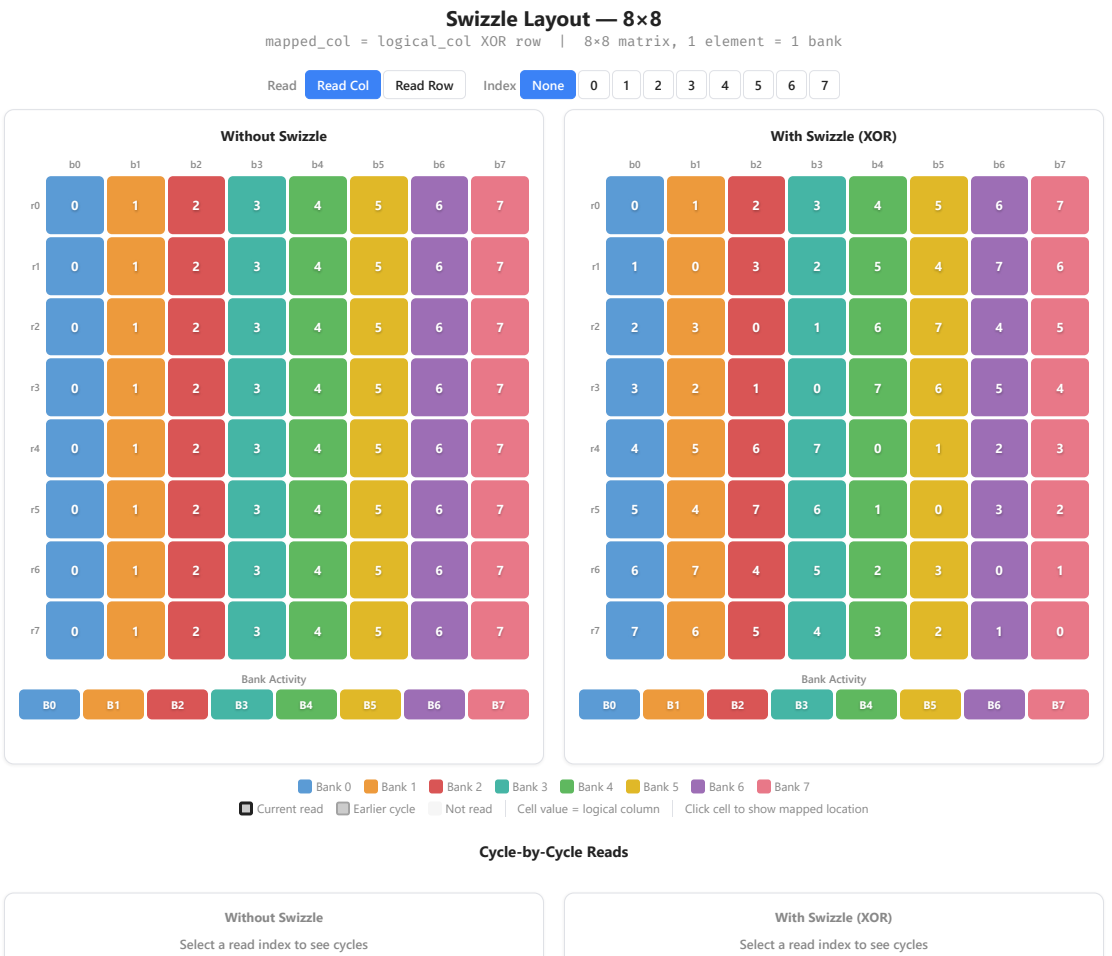
Swizzle Layout

Background: Memory Banks

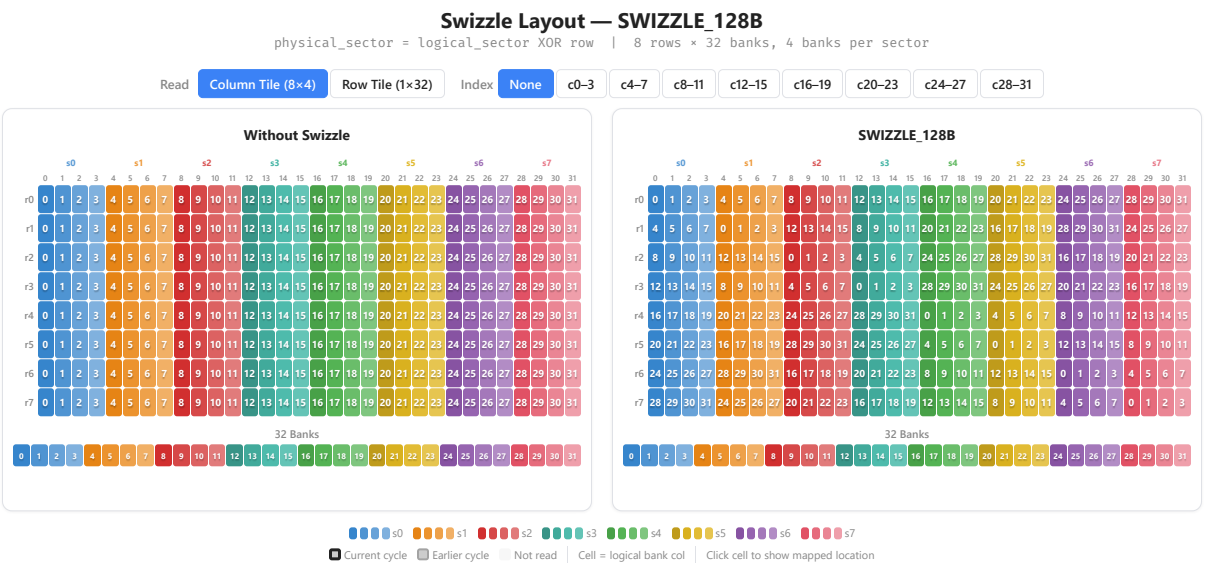
Different memory banks can be read out in parallel



Simple Swizzle: 8×8 Interactive Example



Real Example: SWIZZLE_128B



1 element = 1 bank (4 bytes). A warp of 32 threads reads 32 elements; bank conflicts serialize into extra cycles.
Column Tile (8×4): 8 rows × 4 cols (1 sector) = 32 reads. With swizzle: **1 cycle**. Without: **8 cycles** (8-way conflicts).
Row Tile (1×32): 1 row × 32 cols = 32 reads. Always **1 cycle** (all 32 banks distinct within a row).

Swizzle as a Memory Optimization

Swizzle is an implicit remapping the hardware applies — just enable it

- Pass pre-swizzled addresses and use a consistent swizzle mode (e.g. SWIZZLE_128B) across ops
- Don't worry about swizzle yourself — let hardware handles it.
- Benefit: SWIZZLE_128B → conflict-free access to 8 rows and 8 columns at a time (fp16)

Swizzle is fundamentally a way to enable efficient column access — the idea is not specific to GPU shared memory!